Project Title:
Agora-Link: A Privacy-Preserving AI Compliance Gateway

1. Problem Statement

The rapid institutional adoption of Web3 has exposed a critical "Compliance Gap." Traditional financial infrastructure requires strict Know Your Customer (KYC) and Anti-Money Laundering (AML) oversight. Conversely, public blockchains are inherently transparent, forcing users to expose sensitive transaction histories to meet legal standards. The objective of "Agora-Link" is to bridge Web2 fiat intent with decentralized Web3 settlement, automating regulatory compliance without compromising the user's digital privacy.

Core Implementation (Completed Phase 1):

- **Theme 3 (Crypto & Card Payments):** Successfully bridged the gap between Web2 and Web3 by integrating Stripe fiat tokenization with an Ethereum Solidity Vault (AgoraVault) on the Sepolia testnet.
- **Theme 5 (KYC & AML):** Developed an agentic, AI-driven compliance engine using an Isolation Forest machine learning model to dynamically score and block transactions based on behavioral anomaly detection.

Future Architecture (Phase 2 Roadmap):

- **Theme 1 (Blockchain Privacy):** While the current gateway handles robust AML logic, the architectural foundation has been laid to integrate COTI V2 Garbled Circuits and ZK-Proofs. Phase 2 will transition the compliance checks to a Zero-Knowledge environment, allowing the AI to verify wallet legitimacy without exposing the user's unencrypted transaction history to the gateway.

2. Core Architecture & Execution (63-Hour Sprint)

The prototype utilizes a segmented architecture to decouple Web2 compliance checks from Web3 execution:

- **AI Compliance Engine (Python/Flask/Scikit-learn):** Deployed an Isolation Forest machine learning model to evaluate wallet age and transaction velocity. It dynamically scores and blocks anomalous behavioral patterns with low latency before transactions interact with the blockchain.

- **Web2-to-Web3 Gateway (Node.js/Ethers.js):** Engineered a highly concurrent microservice bridge that securely captures Stripe fiat tokenization webhooks and translates them into secure blockchain state changes.

- **Smart Contract Escrow (Solidity/Hardhat):** Developed and rigorously tested the AgoraVault contract. Deployed on the Sepolia testnet, it acts as a secure escrow that only releases funds upon cryptographic verification from the AI compliance logic.

- **Immutable Auditing (React.js/MongoDB):** Built a responsive, Glassmorphic frontend paired with a real-time MongoDB Atlas admin dashboard to log approved settlements and AI-flagged anomalies off-chain.

3. Challenges Faced & Solutions

- Challenge 1: Web3 Gas Management & State Asynchrony: The backend gateway initially experienced silent 500 server errors during the final Sepolia smart contract payout phase.

  Solution: Implemented verbose simulation logging in Node.js, tracing the failure to an execution reverted: "Insufficient vault balance" error. Resolved this by acting as a temporary liquidity provider, manually injecting test ETH into the AgoraVault via MetaMask to ensure automated settlements could execute without gas failures.

- Challenge 2: Environment Dependency Conflicts: Deep dependency and versioning conflicts within the local EVM environment (Hardhat) threatened to block the strict 10-day development timeline.

  Solution: Pivoted to a hybrid deployment strategy. Offloaded smart contract compilation, testing, and deployment to a cloud-based EVM (Remix IDE). This isolated the local environment strictly for the Node.js/Python API integration, ensuring zero pipeline blockage and maintaining rapid development velocity.

4. Future Improvements: Cryptographic Privacy (Theme 1)

While Phase 1 successfully handles automated, proactive AML logic, the ultimate objective is to achieve total on-chain data minimization. If allocated more time, Phase 2 will focus exclusively on integrating **COTI V2 Garbled Circuits and ZK-SNARKs**.

The current Node.js pipeline has been architected to support this transition into a Zero-Knowledge environment. This upgrade will allow the AI engine to verify wallet legitimacy and validate risk scores without exposing the user's unencrypted transaction history to the gateway or the public ledger.

5. Developer Acceleration & AI Tooling

To maximize development velocity during the strict 10-day sprint, I integrated AI coding assistants (Google Gemini) directly into my VS Code workflow. This was treated strictly as a developer multiplier, rather than a logic replacement:

- **Rapid Prototyping & Boilerplate:** Utilized AI to rapidly scaffold the initial React frontend and Node.js server structures, allowing me to dedicate the bulk of my man-hours to the complex Web3/Stripe bridging logic.

- **Complex Debugging:** Leveraged the LLM as a diagnostic tool to parse cryptic EVM revert codes and React CORS policy conflicts, drastically reducing downtime.

- **Architectural Validation:** Used the AI as a sounding board to evaluate different machine learning algorithms, validating the choice of the *Isolation Forest* model for unlabelled transaction data.

- ***Security Note:*** *While AI was used to accelerate typing and debugging, all core business logic, smart contract access controls, and Ethers.js transaction payloads were manually authored, vetted, and deployed t*economy.